Event Sourcing & CQRS

vent Sourcing/CQRS · Bank account ledger + balance read model · Build with Waq

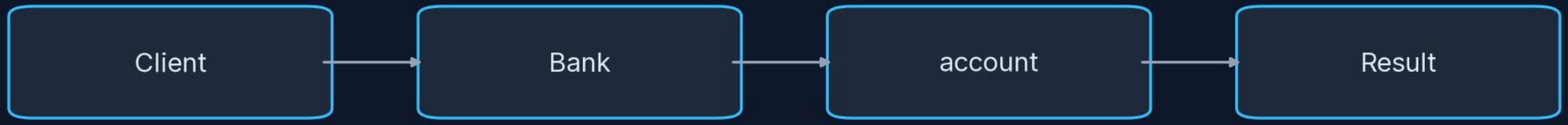
Today's Hook

- Problem: Bank account ledger + balance read model
- Pattern: Event Sourcing/CQRS — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Bank account ledger + balance read model
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Event Sourcing/CQRS: Bank account ledger + balance read model

Real-World Scenario

- Running example: Bank account ledger + balance read model
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing.problem;

import java.util.*;

public final class NaiveAccount {
  private int bal; void deposit(int c){ bal+=c; } void withdraw(int c){ bal-=c; } int bal(){ return bal; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

sealed interface Event permits Deposited, Withdrawn {}
record Deposited(int cents) implements Event {}
record Withdrawn(int cents) implements Event {}
final class Account {
  private final List<Event> events=new ArrayList<>();
  void deposit(int c){ if(c<=0) throw new IllegalArgumentException("deposit"); events.add(new Deposited(c)); }
  void withdraw(int c){ if(c<=0 || balance()<c) throw new IllegalStateException("funds"); events.add(new Withdrawn(c)); }
  int balance(){ int b=0; for(Event e: events){ if(e instanceof Deposited d) b+=d.cents(); else if(e instanceof Withdrawn w) b-=w.cents(); } return b; }
  List<Event> history(){ return List.copyOf(events); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Event Sourcing: bank ledger + balance ===");
    System.out.println("--- naive ---"); var n=new NaiveAccount(); n.deposit(100); n.withdraw(40); System.out.println(n.bal());
System.out.println("--- event sourcing ---");
var a=new Account(); a.deposit(100); a.withdraw(40); System.out.println("bal="+a.balance()); System.out.println("events="+a.history());
try { a.withdraw(1000); } catch(IllegalStateException e){ System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== Event Sourcing: bank ledger + balance ===
--- naive ---
60
--- event sourcing ---
bal=60
events=[Deposited[cents=100], Withdrawn[cents=40]]
caught=funds
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Event Sourcing/CQRS.

Related Patterns

- Contrast Event Sourcing/CQRS with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Event Sourcing/CQRS → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.